

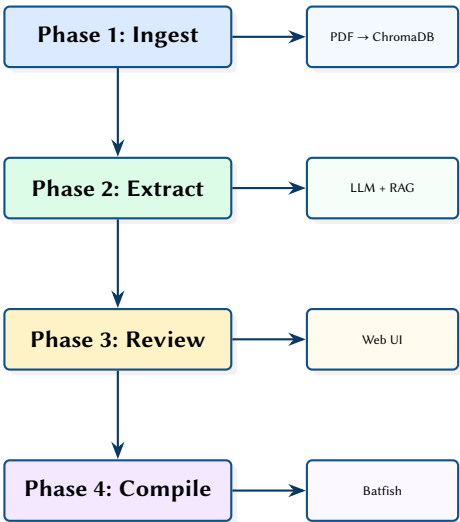
LLM-Powered Network Configuration Parsing and Cross-Vendor Compilation

RAG-Based Extraction, Human-in-the-Loop Review, and Semantic Validation

Simon Knight

March 2026 • Version 1.0.0

Last updated: March 12, 2026



Abstract. This report presents a network automation framework that decouples configuration from vendor-specific syntax using LLM-powered Retrieval-Augmented Generation. The system operates in four phases: vendor documentation ingestion into a ChromaDB vector store, LLM-powered topology extraction with confidence scoring and evidence tracking, human-in-the-loop web-based review with citation enforcement, and cross-vendor configuration compilation with Batfish semantic validation. The vendor-neutral intermediate representation models network relationships (interfaces, VLANs, OSPF, BGP) rather than device syntax, enabling the same topology to compile to Cisco IOS and Arista EOS output with full evidence traceability.

Contents

1	Introduction and Motivation	2
2	Phase 1: Knowledge Base Ingestion	3
3	Phase 2: Topology Extraction	3
4	Phase 3: Human-in-the-Loop Review	6
5	Phase 4: Cross-Vendor Compilation	7
5.1	Semantic Validation	8
6	Design Summary	10
	List of Figures	12
	List of Tables	12
	List of Design Decisions & Rules	13
	Revision History	13
LLM	Large Language Model	2
RAG	Retrieval-Augmented Generation	2
IR	Intermediate Representation	2
HITL	Human-in-the-Loop	6
OSPF	Open Shortest Path First	4
BGP	Border Gateway Protocol	4

1 Introduction and Motivation

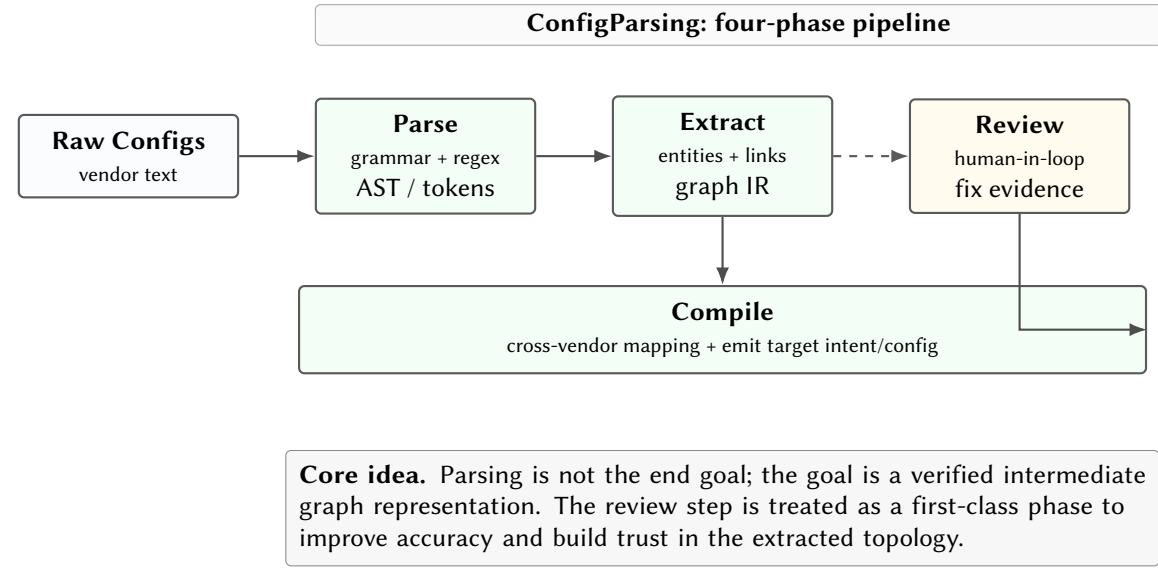


Figure 1. From raw configurations to a validated graph IR and cross-vendor compilation.

Network configuration is inherently vendor-specific: the same logical intent (“interface Ethernet1 has IP 10.0.0.1/30 in VLAN 100”) is expressed differently across Cisco IOS, Arista EOS, Juniper JunOS, and others. Traditional parsers use hand-written regular expressions or TextFSM templates, which are brittle, vendor-locked, and require constant maintenance as CLI syntax evolves.

This framework takes a fundamentally different approach: it uses Large Language Model (LLM)-powered Retrieval-Augmented Generation (RAG) [1] to understand vendor documentation and extract network intent from configurations, producing a vendor-neutral Intermediate Representation (IR) that can be compiled to any target vendor.

: Evidence-Backed Extraction

Every extracted field carries a confidence score (0.0–1.0) and evidence citations linking the value to source configuration lines and documentation chunks. Low-confidence extractions are automatically flagged for human review.

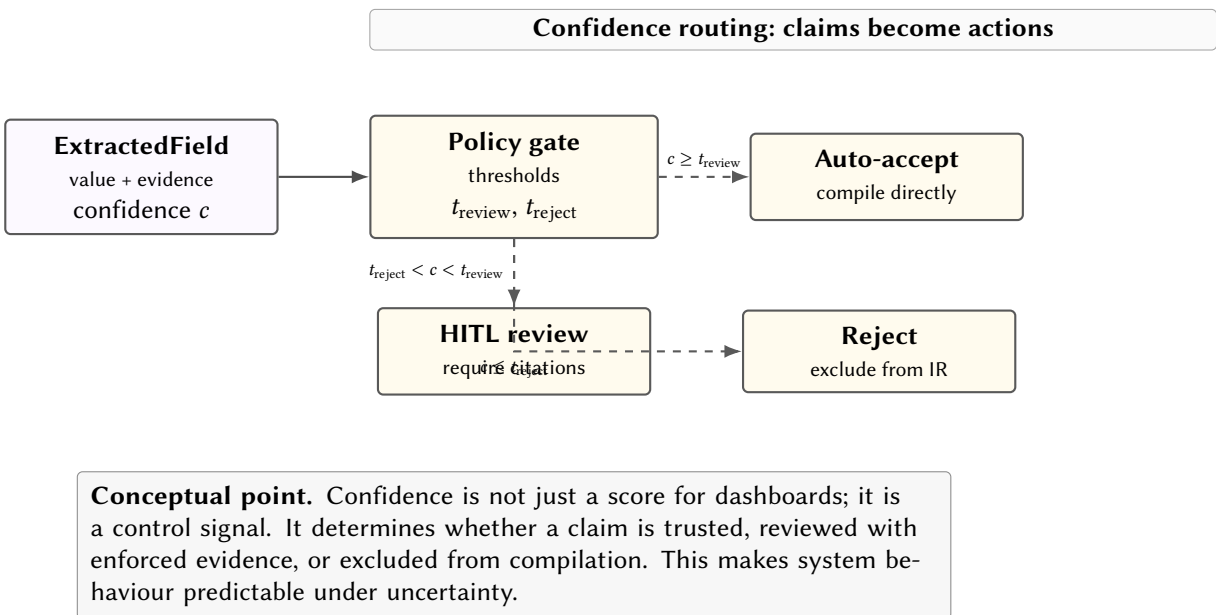


Figure 2. Confidence routing converts extraction uncertainty into operational behaviour: accept, review, or reject with explicit thresholds.

2 Phase 1: Knowledge Base Ingestion

Vendor documentation (PDF manuals, CLI references) is ingested through a layout-aware PDF parser that preserves section structure and code blocks. Parsed chunks are embedded and stored in ChromaDB [2] with deterministic chunk IDs for reproducible citations.

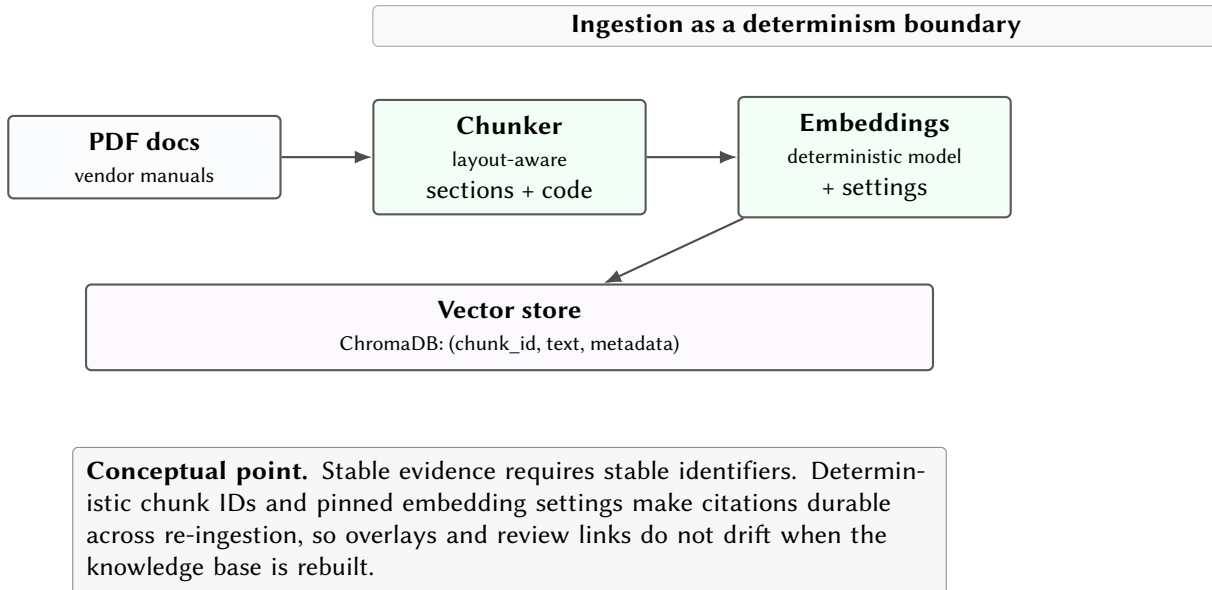
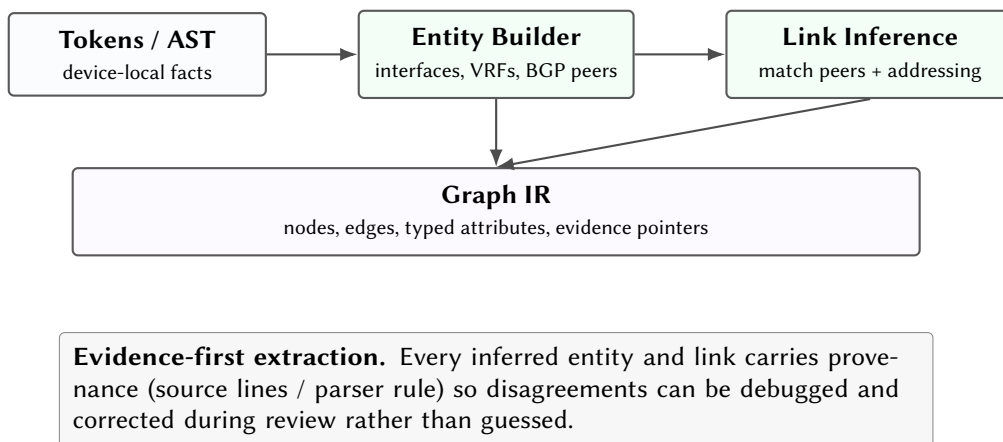


Figure 3. Knowledge-base ingestion as a reproducibility boundary. Deterministic chunk IDs preserve end-to-end evidence chains across re-ingestion.

```
class DocumentIngester:
    def ingest_pdf(self, path: Path, vendor: str) -> int:
        chunks = self.parser.extract_chunks(path)
        for chunk in chunks:
            chunk_id = deterministic_id(vendor, path.name, chunk.page, chunk.index)
            self.collection.add(
                ids=[chunk_id],
                documents=[chunk.text],
                metadatas=[{"vendor": vendor, "page": chunk.page}],
            )
        return len(chunks)
```

3 Phase 2: Topology Extraction



Insight:
Deterministic chunk IDs ensure that re-ingesting the same document produces identical identifiers, preserving evidence citations across extractions without invalidating the overlay database.

Figure 4. Topology extraction as a graph construction problem: facts are lifted into entities, then joined into links with explicit provenance.

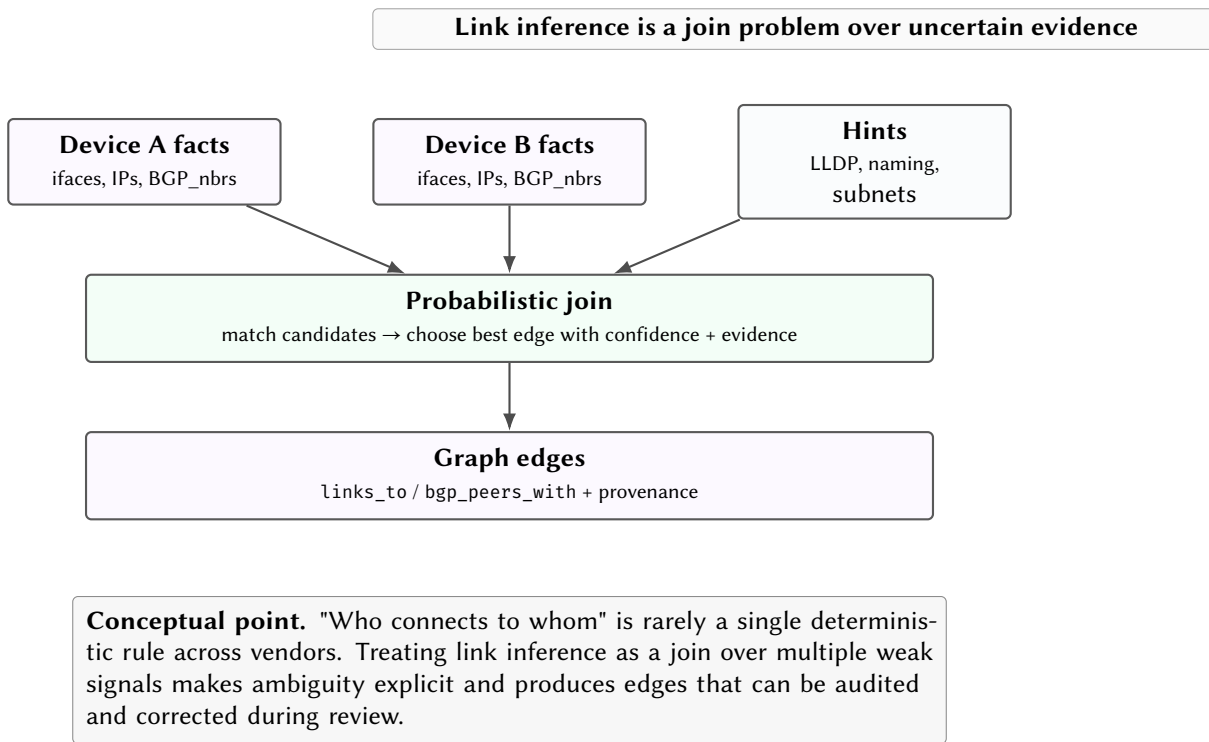


Figure 5. Link inference as a constrained join. Multiple evidence sources contribute to candidate matches; the chosen edge carries confidence and provenance.

The extraction engine processes vendor configurations through an LLM with RAG context from the knowledge base. For each configuration section, relevant documentation chunks are retrieved from ChromaDB and included in the prompt, giving the LLM vendor-specific context for accurate parsing.

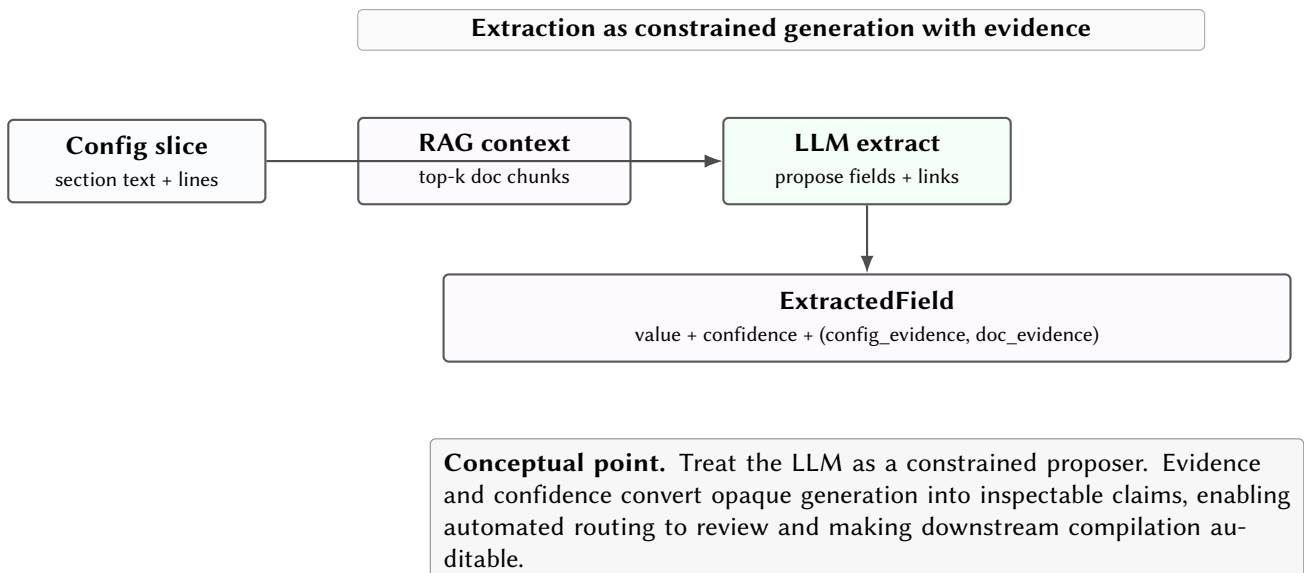


Figure 6. Evidence-backed extraction. Config text and retrieved docs jointly condition the LLM, which outputs values plus provenance, not just a best-guess parse.

The output is a vendor-neutral IR modelling:

- **Interfaces** — name, IP addresses, VLANs, status
- **Routing** — Open Shortest Path First (OSPF) areas, Border Gateway Protocol (BGP) peers, static routes
- **Services** — DHCP, NTP, DNS, ACLs

Each extracted field includes:

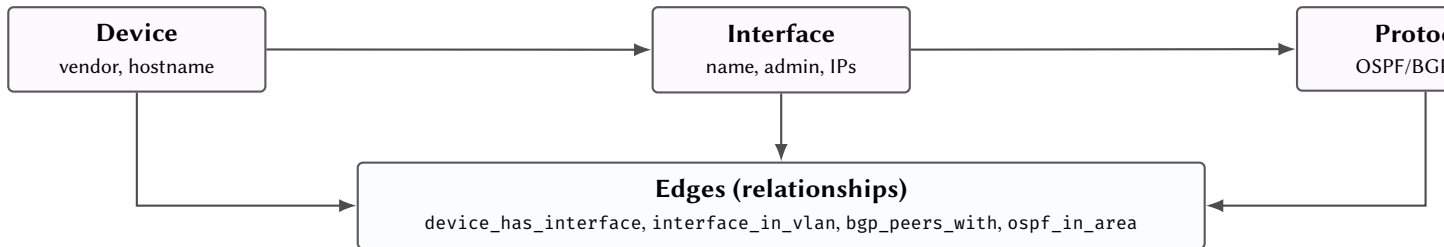
```
@dataclass
class ExtractedField:
```

```

value: Any
confidence: float # 0.0 - 1.0
config_evidence: list[str] # Source config line numbers
doc_evidence: list[str] # ChromaDB chunk IDs

```

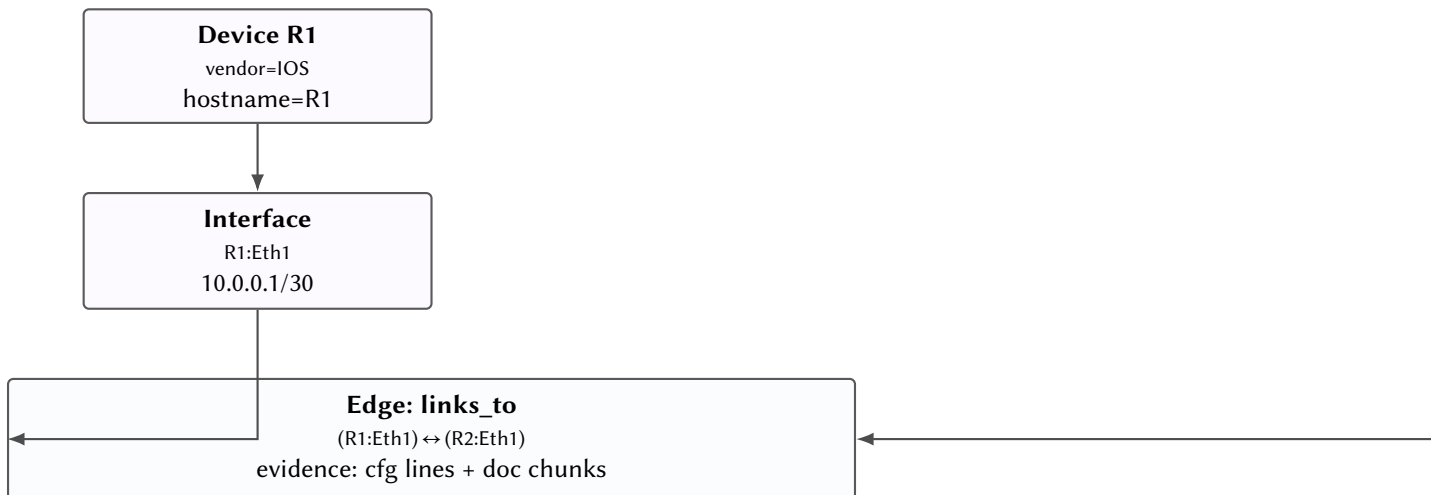
The IR is a typed graph schema (not a pile of strings)



Conceptual point. A typed graph encodes intent as relationships. Compilation and validation operate on this structure (joins, constraints, reachability), which is why the system generalises across vendors better than regex-only parsing.

Figure 7. IR as a schema: entities plus typed relationships. Evidence attaches to nodes/edges, enabling traceable compilation and semantic checks.

Concrete IR example: 2-router mini-topology with evidence pointers



Conceptual point. Even a tiny topology illustrates the advantage of a graph IR: the link is a first-class relationship object with its own provenance, and both vendor configs compile from the same intent-level representation.

Figure 8. A minimal graph IR instance. Devices, interfaces, and a link edge are represented explicitly; evidence attaches at the edge/field level for auditability.

Design Decision 1: Hallucination Protection

Fields extracted with confidence below the configurable threshold (default 0.6) are automatically flagged for Human-in-the-Loop (HITL) review. The evidence chain (config line → doc chunk → extracted value) enables reviewers to verify the LLM's reasoning rather than trusting the output blindly.

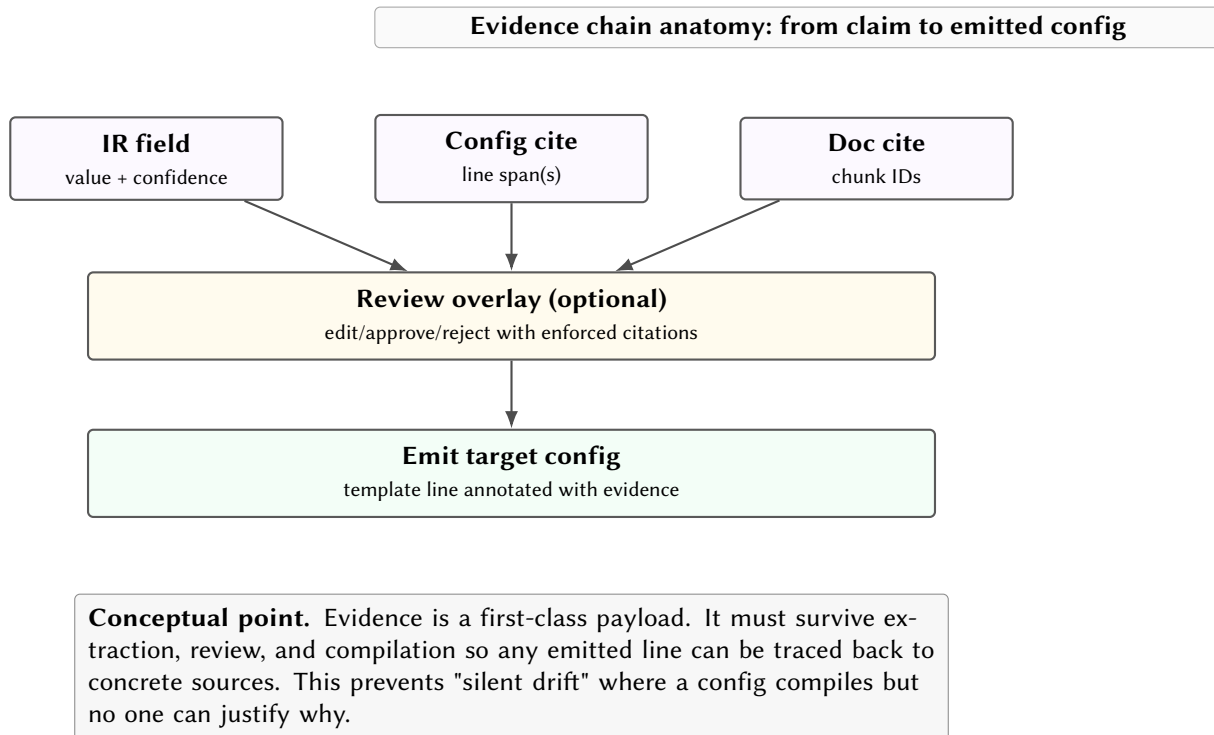


Figure 9. End-to-end evidence chain. The same provenance objects support auditing, review enforcement, and annotated compilation outputs.

4 Phase 3: Human-in-the-Loop Review

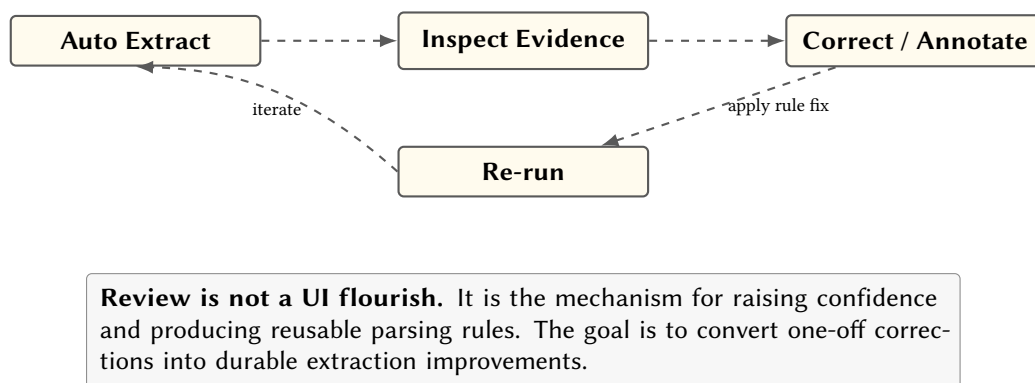


Figure 10. Human-in-the-loop review workflow: evidence inspection leads to corrections that feed back into improved extraction rules.

A web-based review interface presents flagged extractions with their evidence chains. Reviewers can:

- **Approve** — Accept the extraction as-is (confidence set to 1.0).
- **Edit** — Modify the value with mandatory evidence citation.
- **Reject** — Mark as incorrect; excluded from compilation.

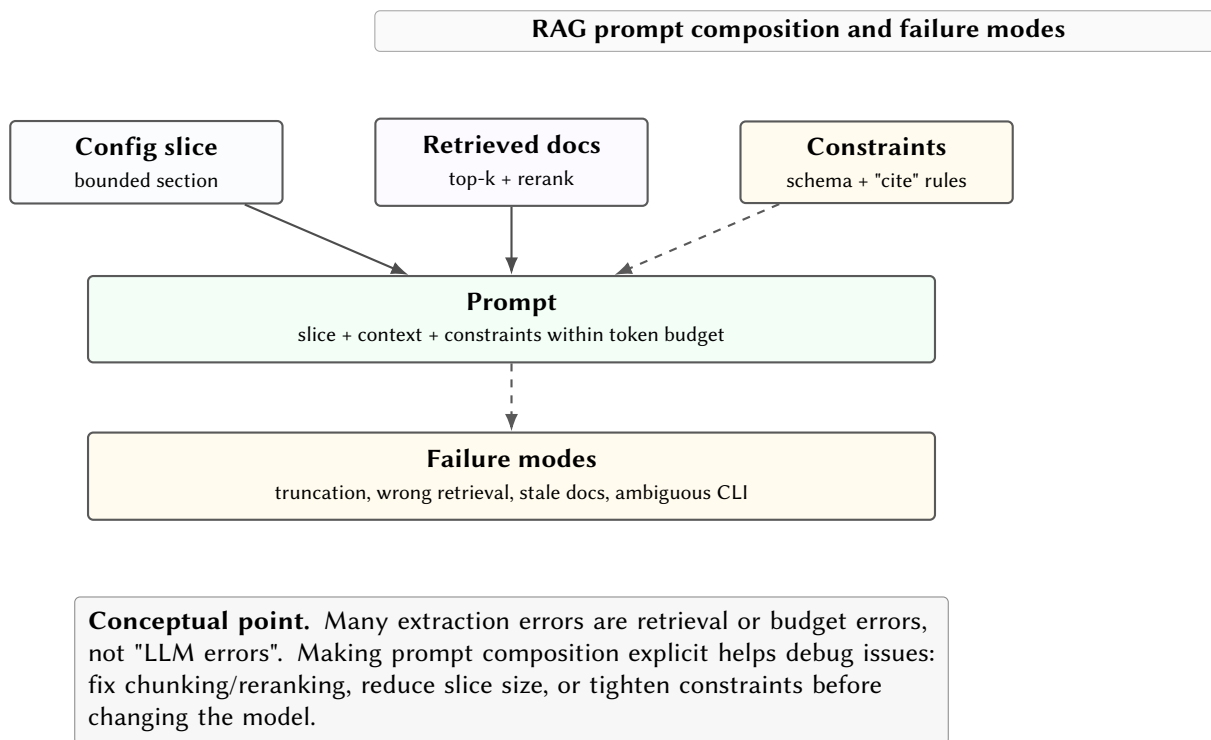


Figure 11. Prompt composition is an engineering surface. Retrieval quality and token budget management strongly shape extraction accuracy and explainability.

Approved edits are stored as overlays that persist across re-extractions. When the extraction pipeline runs again (e.g. after documentation updates), approved overlays are applied automatically, preserving human corrections.

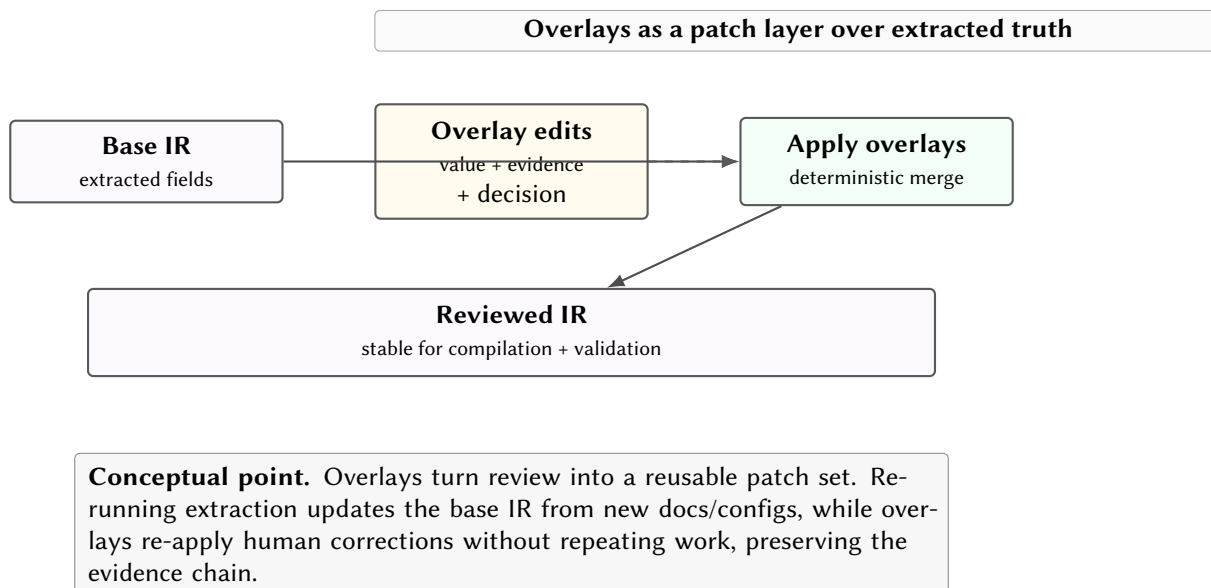


Figure 12. HITL review produces overlays: durable patches applied deterministically on top of regenerated extraction output.

: Evidence Enforcement

All manual edits must cite either a configuration line number or a documentation chunk ID. This prevents unsupported assertions from entering the IR and maintains the end-to-end evidence chain.

5 Phase 4: Cross-Vendor Compilation

The compiler transforms the vendor-neutral IR into target vendor configurations using Jinja2 templates. Currently supported targets are Cisco IOS and Arista EOS.

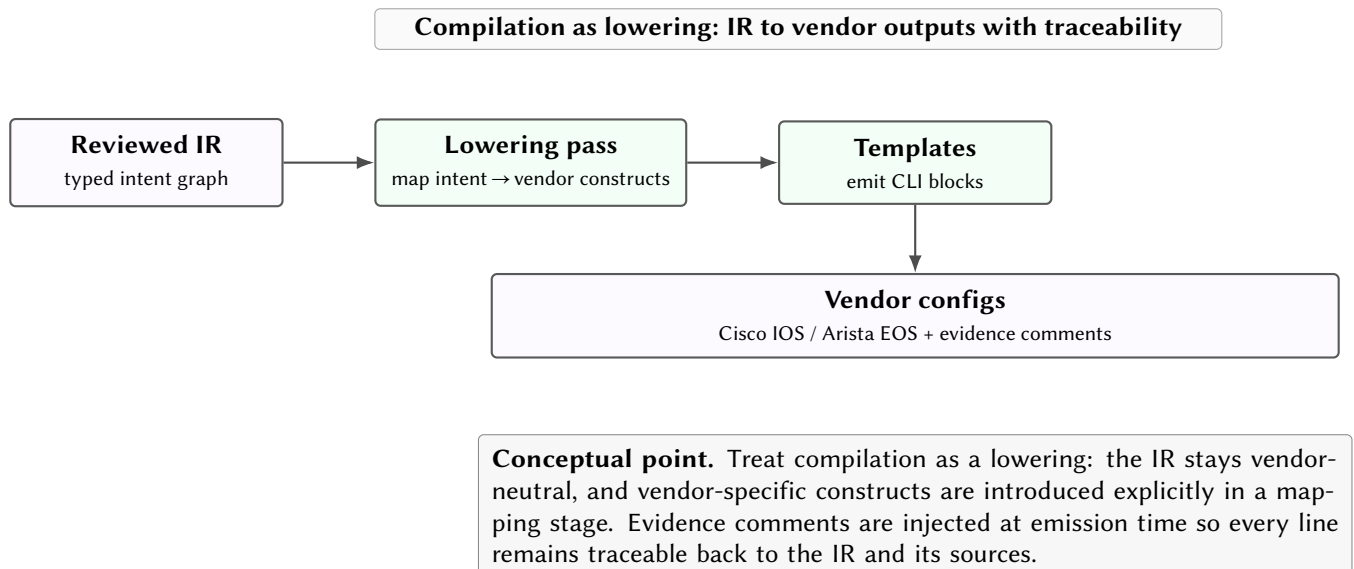


Figure 13. Cross-vendor compilation as a lowering pipeline. Vendor constructs are introduced explicitly (mapping), then rendered (templates) with evidence annotations.

Generated configurations include evidence comments tracing each line back to the IR field and its original source:

```

! Source: interface Ethernet1 (confidence: 0.95)
! Evidence: config line 42, doc chunk cisco-ios-ref:p234:3
interface Ethernet1
  ip address 10.0.0.1 255.255.255.252
  no shutdown
  
```

5.1 Semantic Validation

Compiled configurations are validated using Batfish [3], which performs semantic analysis to verify:

- Reachability between expected endpoint pairs
- Route table correctness (expected prefixes present)
- ACL behaviour matches intent
- No unintended route leaking between VRFs

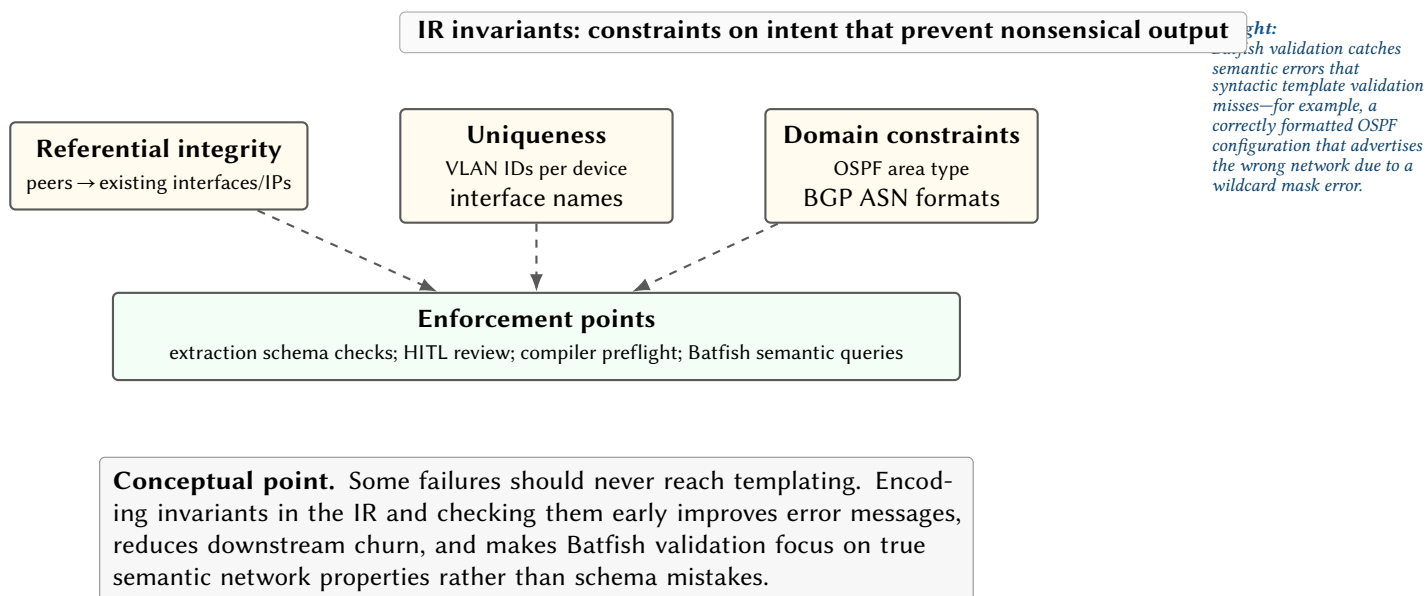


Figure 14. IR constraints and where they are enforced. Invariants provide early failure and clearer debugging than post-hoc syntax checks.

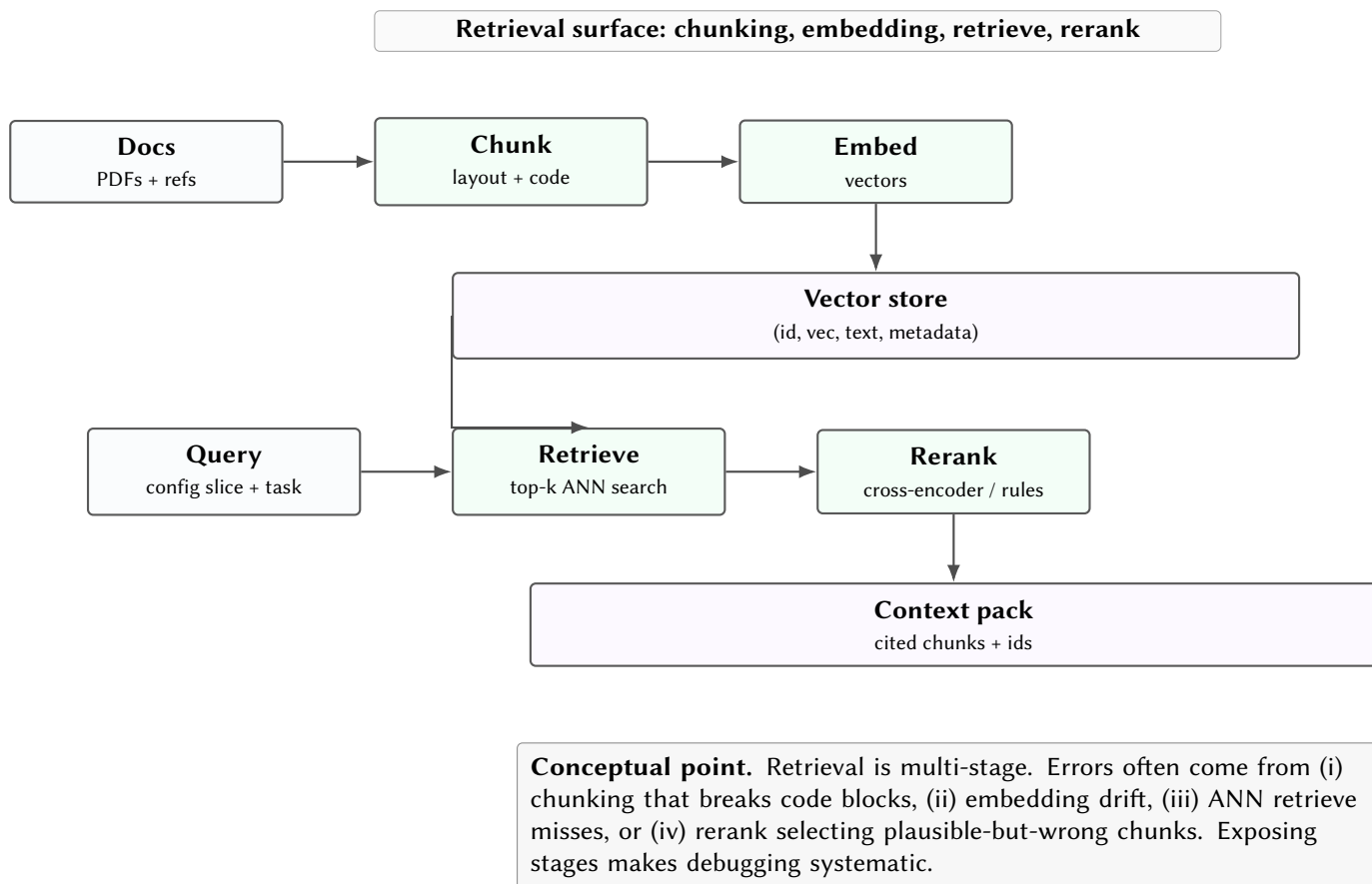


Figure 15. RAG retrieval surface. Chunking and reranking are first-class levers for accuracy and explainability.

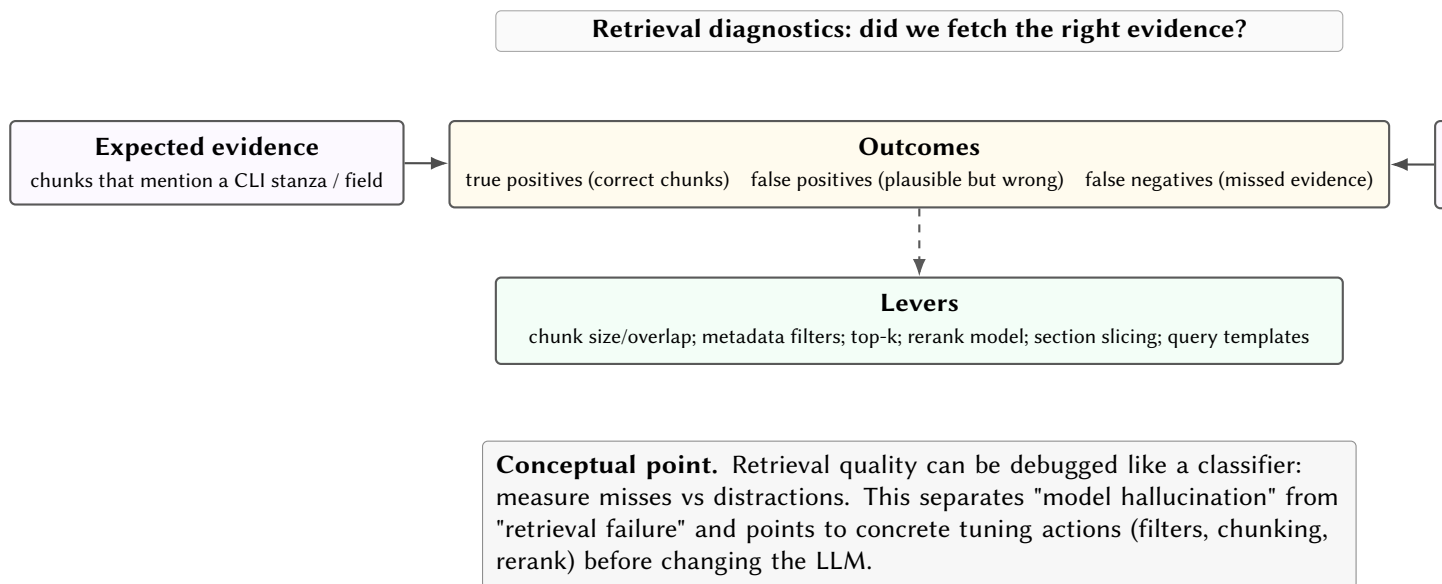


Figure 16. A practical mental model for RAG tuning. Treat retrieval as a classification problem over evidence chunks and tune for fewer false negatives (missed docs) and false positives (distractors).

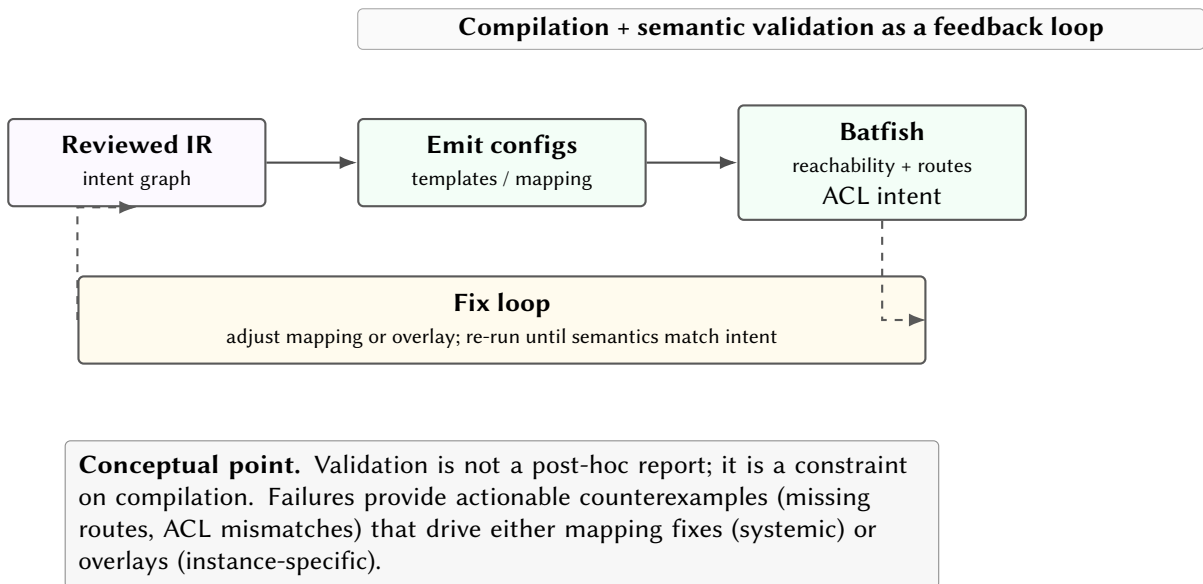


Figure 17. Semantic validation closes the loop. Batfish turns emitted configs into semantic facts, which must satisfy the IR’s intent.

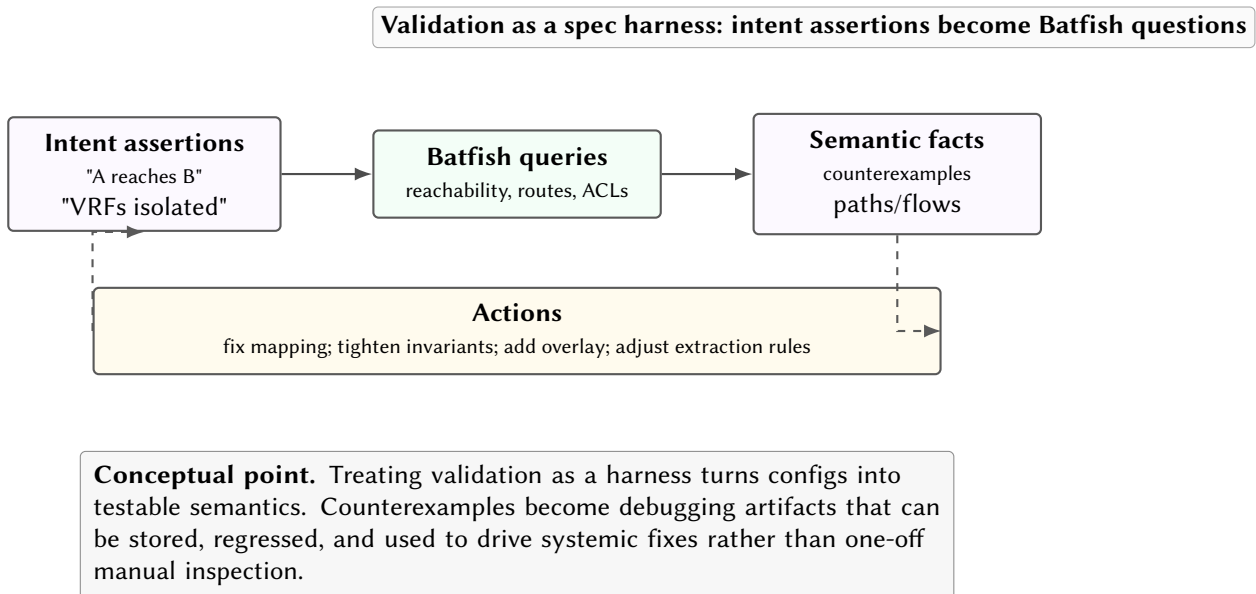


Figure 18. Batfish as a semantic test harness. Intent assertions compile into queries; counterexamples drive iterative improvements.

6 Design Summary

Table 1. Key design decisions.

Decision	Rationale
RAG over fine-tuning	Vendor docs change frequently; RAG adapts without retraining
Confidence scoring	Quantifies extraction reliability; enables automatic HITL routing
Deterministic chunk IDs	Stable citations across re-ingestion; overlays survive doc updates
Overlay persistence	Human corrections survive re-extraction without manual re-review
Batfish validation	Semantic verification catches errors invisible to syntactic checks

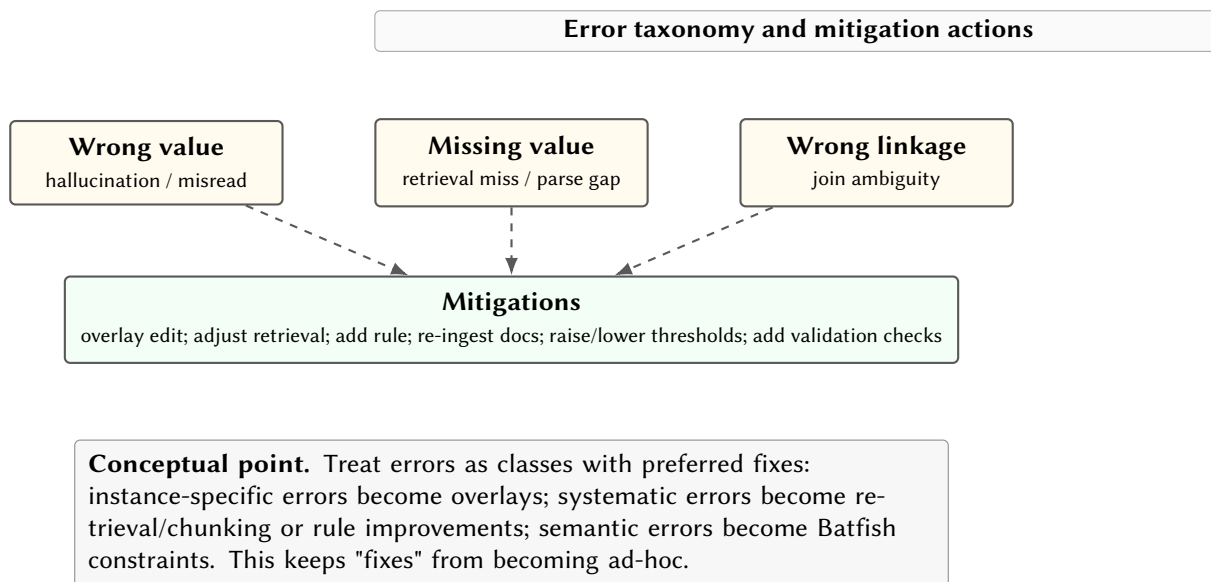


Figure 19. Operational taxonomy: common failure modes and the most effective mitigation levers in the pipeline.

References

- [1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," *NeurIPS*, 2020.
- [2] Chroma, *ChromaDB: The open-source embedding database*, 2024. [Online]. Available: <https://www.trychroma.com>
- [3] Intentionet, *Batfish: Network configuration analysis*, 2024. [Online]. Available: <https://www.batfish.org>

List of Figures

1	From raw configurations to a validated graph IR and cross-vendor compilation.	2
2	Confidence routing converts extraction uncertainty into operational behaviour: accept, review, or reject with explicit thresholds.	2
3	Knowledge-base ingestion as a reproducibility boundary. Deterministic chunk IDs preserve end-to-end evidence chains across re-ingestion.	3
4	Topology extraction as a graph construction problem: facts are lifted into entities, then joined into links with explicit provenance.	3
5	Link inference as a constrained join. Multiple evidence sources contribute to candidate matches; the chosen edge carries confidence and provenance.	4
6	Evidence-backed extraction. Config text and retrieved docs jointly condition the LLM, which outputs values plus provenance, not just a best-guess parse.	4
7	IR as a schema: entities plus typed relationships. Evidence attaches to nodes/edges, enabling traceable compilation and semantic checks.	5
8	A minimal graph IR instance. Devices, interfaces, and a link edge are represented explicitly; evidence attaches at the edge/field level for auditability.	5
9	End-to-end evidence chain. The same provenance objects support auditing, review enforcement, and annotated compilation outputs.	6
10	Human-in-the-loop review workflow: evidence inspection leads to corrections that feed back into improved extraction rules.	6
11	Prompt composition is an engineering surface. Retrieval quality and token budget management strongly shape extraction accuracy and explainability.	7
12	HITL review produces overlays: durable patches applied deterministically on top of regenerated extraction output.	7
13	Cross-vendor compilation as a lowering pipeline. Vendor constructs are introduced explicitly (mapping), then rendered (templates) with evidence annotations.	8
14	IR constraints and where they are enforced. Invariants provide early failure and clearer debugging than post-hoc syntax checks.	8
15	RAG retrieval surface. Chunking and reranking are first-class levers for accuracy and explainability.	9
16	A practical mental model for RAG tuning. Treat retrieval as a classification problem over evidence chunks and tune for fewer false negatives (missed docs) and false positives (distractors).	9
17	Semantic validation closes the loop. Batfish turns emitted configs into semantic facts, which must satisfy the IR's intent.	10
18	Batfish as a semantic test harness. Intent assertions compile into queries; counterexamples drive iterative improvements.	10
19	Operational taxonomy: common failure modes and the most effective mitigation levers in the pipeline.	11

List of Tables

1	Key design decisions.	10
---	-------------------------------	----

List of Design Decisions & Rules

1	Design Rule: Evidence-Backed Extraction	2
1	Hallucination Protection	6
2	Design Rule: Evidence Enforcement	7

Revision History

Version	Date	Changes
1.0.0	March 2026	Initial report (all four phases complete)